

(8) 帮助桌面或用户支持热线部门。

(9) 配置管理部门。

变更控制委员会应该有一个总则，用于描述变更控制委员会的目的、授权范围、成员构成、做出决策的过程及操作步骤。总则也应该说明举行会议的频度和事由。管理范围描述该委员会能做什么样的决策，以及有哪一类决策应上报到高一级的委员会。过程及操作步骤如下：

(1) 制定决策。制定决策过程的描述应确认：

①变更控制委员会必须到会的人数或做出有效决定必须出席的人数。

②决策的方法（例如投票、一致通过或其他机制）。

③变更控制委员会主席是否可以否决该集体的决定。

变更控制委员会应该对每个变更权衡利弊后做出决定。“利”包括节省的资金或额外的收入、增强的客户满意度、竞争优势、减少上市时间；“弊”是指接受变更后产生的负面影响，包括增加的开发费用、推迟的交付日期、产品质量的下降、减少的功能、用户不满意度。

(2) 交流情况。一旦变更控制委员会做出决策，指派的人员应及时更新请求的状态。

(3) 重新协商约定。变更总是有代价的，即使拒绝的变更也会因为决策行为（提交、评估、决策）而耗费资源。当项目接受了重要的需求变更时，为了适应变更情况要与管理部和客户重新协商约定。协商的内容可能包括推迟交货时间、要求增加人手、推迟实现尚未实现的较低优先级的需求，或者质量上进行折中。

11.8.2 需求跟踪

需求跟踪包括编制每个需求同系统元素之间的联系文档，这些元素包括其他需求、体系结构、其他设计部件、源代码模块、测试、帮助文件和文档等，这是为了在整个项目的工件之间形成水平可追踪性。跟踪能力信息使变更影响分析十分便利，有利于确认和评估实现某个建议的需求变更所必需的工作。

需求跟踪提供了由需求到产品实现整个过程范围的明确查阅能力。需求跟踪的目的是建立与维护“需求-设计-编程-测试”之间的一致性，确保所有的工作成果符合用户需求。

需求跟踪有两种方式：

(1) 正向跟踪。检查产品需求规格说明书中的每个需求是否都能在后继工作成果中找到对应点。

(2) 逆向跟踪。检查设计文档、代码、测试用例等工作成果是否都能在产品需求规格说明书中找到出处。

正向跟踪和逆向跟踪合称为双向跟踪。不论采用何种跟踪方式，都要建立与维护需求跟踪矩阵（即表格）。需求跟踪矩阵保存了需求与后继工作成果的对应关系。

跟踪能力是优秀需求规格说明书的一个特征，为了实现可跟踪能力，必须统一地标识出每一个需求，以便能明确地进行查阅。

需求跟踪是个要求手工操作且劳动强度很大的任务，要求组织提供支持。随着系统开发的进行和维护的执行，要保持关联信息与实际一致。跟踪能力信息一旦过时，可能再也不会重建它。在实际项目中，往往采用专门的配置管理工具来实现需求跟踪。

第 12 章 软件架构设计

传统的软件开发过程可以划分为从概念到实现的若干个阶段，包括软件计划、需求分析、软件设计、软件实现和软件测试等。在这种开发过程中，如何将需求分析的成果转换为软件设计，这个问题一直困扰着研究人员和实践工作者。近年来，软件工程界提出了各种需求工程和软件建模技术，然而，在软件需求和设计之间仍然存在一条很难逾越的鸿沟，从而很难有效地将需求转换为相应的设计。为此，学者们提出了软件架构（Software Architecture）的概念，并试图在软件需求与设计之间架起一座桥梁，重点解决系统结构和需求向实现平坦过渡的问题。

另一方面，随着软件系统规模越来越大、越来越复杂，整个系统的结构和规格说明显得越来越重要。在这种背景下，人们也逐渐认识到软件架构的重要性，促进了软件架构技术的快速发展和应用。

12.1 软件架构概述

软件架构为软件系统提供了一个结构、行为和属性的高级抽象，由构件的描述、构件的相互作用（连接件）、指导构件集成的模式以及这些模式的约束组成。软件架构指定了系统的组织结构和拓扑结构，并且显示了系统需求和构件之间的对应关系，提供了一些设计决策的基本原理。

软件架构虽脱胎于软件工程，但其形成也借鉴了计算机架构和网络架构中很多宝贵的思想和方法。近年来，软件架构已完全独立于软件工程，成为计算机科学的一个最新的研究方向和独立学科分支。软件架构研究的主要内容涉及软件架构描述、软件架构风格、软件架构评估和软件架构的形式化方法等。解决好软件的复用、质量和维护问题，是研究软件架构的根本目的。

1. 软件架构的意义

对于软件项目的开发来说，一个清晰的软件架构是首要的。鉴于架构的重要性，Perry 将软件架构视为软件开发中第一类重要的设计对象，Barry Boehm 也明确指出：“在没有设计出架构及其规则时，那么整个项目不能继续下去，而且架构应该看作是软件开发中可交付的中间产品”。由此可见，架构在软件开发中为不同的人员提供了共同的交流语言，体现并尝试了系统早期的设计决策，并作为系统设计的抽象，为实现框架和构件的共享和重用、基于架构的软件开发提供了有力的支持。

（1）架构是项目干系人进行交流的手段。软件架构代表了系统的高层抽象，不同的项目干系人关心着系统的不同方面，而这些方面都受架构的影响，因此，架构可能是所有项目干系人共同关心的一个重要因素。例如，用户关心系统是否满足可用性和可靠性需求；客户关心的是系统能否在规定时间内完成，并且开支在预算范围内；管理人员担心在经费支出和进度条件下，按此架构能否使开发团队成员在一定程度上独立开发，各部分的交互是否遵循统一的规范，开发进度是否可控；开发人员关心的是如何才能实现架构的各项目标。

(2) 架构是早期设计决策的体现。软件架构体现了系统最早的一组设计决策，这些早期的约束比起以后的开发、设计、编码或运行及维护阶段的工作重要得多，对系统生命周期的影响也大得多。早期决策的正确性最难以保证，而且这些决策也最难以改变，影响范围也最大。

(3) 架构明确了对系统实现的约束条件。所谓“实现”就是要用实体来显示出一个软件架构，即要符合架构所描述的结构设计决策，分割成规定的构件，按规定方式互相交互。在具体实现时，必须按照架构的设计，将系统分成若干个组成部分，各部分必须按照预定的方式进行交互，而且每个部分也必须具有架构中所规定的外部特征。这些约束是在系统级或项目范围内作出的，每个构件上工作的实现者是看不见的。这样一来，可以分离关注点，架构设计师不必是算法设计者或精通编程语言，他们只需重点考虑系统的总体权衡（tradeoff）问题，而构件的开发人员在架构给定的约束下进行开发。

(4) 架构决定了开发和维护组织的组织结构。架构包含了对系统的最高层次的分解，因此一般被作为任务划分结构的基础。任务划分结构又规定了计划、调度及预算的单位，决定了开发小组内部交流的渠道、配置控制和文件系统的组织、集成与测试计划和过程等。各开发小组按照架构中对各主要构件接口的规定进行交流。一旦进入维护阶段，维护活动也会反映出软件架构，常由不同的小组分别负责对各具体部分的维护。

(5) 架构制约着系统的质量属性。小的软件系统可以通过编程或调试措施来达到质量属性的要求，而随着软件系统规模的扩大，这种技巧也将越来越无法满足要求。因为在大型软件系统中，质量属性更多的是由系统结构和功能划分来实现的，而不再主要依靠所选用的算法或数据结构。可以使用对架构的评价来预测系统未来的质量属性，架构评估技术可以对按某架构开发出来的软件产品的质量及缺陷做出比较准确的预测。

(6) 架构使推理和控制更改更简单。在整个软件生命周期内，每个架构都将更改划分为三类，分别是局部的、非局部的和架构级的变更。局部变更是最经常发生的，也是最容易进行的，只需修改某一个构件就可以实现。非局部变更的实现则需对多个构件进行修改，但并不改动软件架构。架构级的变更是指会影响各部分的相互关系，甚至要改动整个系统。所以，一个优秀的架构应该能使更改简单易行。

(7) 架构有助于循序渐进的原型设计。一旦确定了架构，就可以对它进行分析，并将它按可执行模型来构造原型，以减少项目开发中的潜在风险。

(8) 架构可以作为培训的基础。在对项目组新成员介绍所开发的系统时，可以首先介绍系统的架构，以及对构件之间如何交互从而实现系统需求的高层次的描述，让项目新成员能很快进入角色。

(9) 架构是可传递和可复用的模型。软件架构体现了一个相对来说比较小又可理解的模型。软件架构级的复用意味着架构的决策能在具有相似需求的多个系统中发生影响，这比代码级的复用要有更大的好处。通过对架构的抽象，架构设计师能够对一些经过实践证明是非常有效的架构进行复用，从而提高设计的效率和可靠性。

2. 软件架构的发展史

在软件系统规模迅速增大的同时，软件开发方法也经历了一系列的变革。在此过程中，软

件架构也由最初模糊的概念发展到一个渐趋成熟的理论和技术。

20世纪70年代以前，尤其是在以ALGOL 68为代表的高级语言出现以前，软件开发基本上都是汇编程序设计，此阶段系统规模较小，很少明确考虑系统结构，一般不存在系统建模工作。20世纪70年代中后期，由于结构化开发方法的出现与广泛应用，软件开发中出现了概要设计与详细设计，其主要任务是数据流设计与控制流设计，此时软件结构已作为一个明确的概念出现在系统开发中。

20世纪80年代初到90年代中期，是OO方法兴起与成熟阶段。由于对象是数据与基于数据之上操作的封装。因此，在OO方法下，数据流设计与控制流设计统一为对象建模，同时，OO方法还提出了一些其他的结构视图。例如，OMT方法提出了功能视图、对象视图和动态视图，Booch方法提出了类图、对象图、状态迁移图、交互图、模块图和进程图，UML则从功能模型、静态模型、动态模型和配置模型等方面描述应用系统的结构。

20世纪90年代以后，是基于构件的软件开发阶段。该阶段以过程为中心，强调软件开发采用构件化技术和架构技术，要求开发出的软件具备很强的自适应性、互操作性、可扩展性和可复用性。此阶段中，软件架构已经作为一个明确的文档和中间产品存在于软件开发过程中。同时，软件架构作为一门学科逐渐得到人们的重视，并成为软件工程领域的研究热点。

纵观软件架构技术的发展过程，从最初的无架构设计到现行的基于架构的软件开发，经历了4个阶段：

- (1) 无架构设计阶段。以汇编语言进行小规模应用程序开发为特征。
- (2) 萌芽阶段。出现了程序结构设计主题，以控制流图和数据流图构成软件结构为特征。
- (3) 初级阶段。出现了从不同侧面描述系统的结构模型，以UML为典型代表。

(4) 高级阶段。以描述系统的高层抽象结构为中心，不关心具体的建模细节，划分了架构模型与传统软件结构的界限。该阶段以Kruchten提出的“4+1”模型为标志。有关该模型的详细知识将在12.2节中介绍。

12.2 软件架构建模

软件架构设计的首要问题是如何表示软件架构，即如何对软件架构建模。根据建模的侧重点不同，可以把软件架构的模型分为5种，分别是结构模型、框架模型、动态模型、过程模型和功能模型。

(1) 结构模型。这是一种最直观和最普遍的建模方法，它以构件、连接件和其他概念来刻画架构，并力图通过架构来反映系统的重要语义内容，包括系统的配置、约束、隐含的假设条件、风格和性质等。研究结构模型的核心是架构描述语言。

(2) 框架模型。框架模型与结构模型类似，但它不太侧重描述结构的细节而更侧重于整体结构。框架模型主要以一些特殊的问题为目标建立只针对和适应该问题的架构。

(3) 动态模型。动态模型是对结构模型或框架模型的补充，研究系统的粗粒度行为性质。例如，描述系统的重新配置或演化等，这类系统通常是激励型的。

(4) 过程模型。过程模型研究构建系统的步骤和过程。

(5) 功能模型。功能模型认为架构是由一组功能构件按层次组成的，下层向上层提供服务。功能模型可以看作是一种特殊的框架模型。

在上述5种模型中，最常用的是结构模型和动态模型。这5种模型各有所长，将它们有机地统一在一起，形成一个完整的模型来刻画软件架构更合适。例如，Kruchten在1995年提出了一个“4+1”的视图模型。“4+1”视图模型从5个不同的视角来描述软件架构，每个视图只关心系统的一个侧面，5个视图结合在一起才能反映软件架构的全部内容。“4+1”视图模型如图12-1所示。

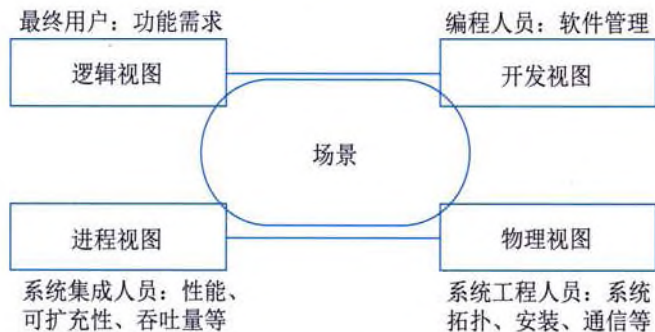


图 12-1 “4+1” 视图模型

(1) 逻辑视图。逻辑视图主要支持系统的功能需求，即系统提供给最终用户的服务。在逻辑视图中，系统分解成一系列的功能抽象，这些抽象主要来自问题领域。这种分解不但可以用来进行功能分析，而且可用作标识在整个系统的各个不同部分的通用机制和设计元素。在OO技术中，通过抽象、封装和继承，可以用对象模型来代表逻辑视图，用类图来描述逻辑视图。逻辑视图中使用的风格为面向对象的风格，在设计中要注意保持一个单一的、内聚的对象模型贯穿整个系统。

(2) 开发视图。开发视图也称为模块视图，在UML中被称为实现视图，它主要侧重于软件模块的组织和管理。开发视图要考虑软件内部的需求，例如，软件开发的容易性、软件复用和软件的通用性，要充分考虑到由于具体开发工具的不同而带来的局限性。开发视图通过系统I/O关系的模型图和子系统图来描述。

(3) 进程视图。进程视图侧重于系统的运行特性，主要关注一些非功能性需求，例如，系统的性能和可用性等。进程视图强调并发性、分布性、系统集成性和容错能力，以及逻辑视图中的功能抽象如何适合进程结构等，它也定义了逻辑视图中的各个类的操作具体是在哪一个线程中被执行的。进程视图可以描述成多层抽象，每个级别分别关注不同的方面。

(4) 物理视图。物理视图在UML中被称为部署视图，主要考虑如何把软件映射到硬件上，它通常要考虑到解决系统拓扑结构、系统安装和通信等问题。当软件运行于不同的物理节点上时，各视图中的构件都直接或间接地对应于系统的不同节点上。因此，从软件到节点的映射要有较高的灵活性，当环境改变时，对系统其他视图的影响最小化。

(5) 场景。场景可以看作是那些重要系统活动的抽象，它使4个视图有机联系起来，从某种意义上说场景是最重要的需求抽象。场景视图对应UML中的用例视图。在开发软件架构时，它可以帮助架构设计师找到构件及其相互关系。同时，架构设计师也可以用场景来分析一个特